

TECNICATURA UNIVERSITARIA EN PROGRAMACIÓN
Universidad Tecnológica Nacional

Programación 2

Trabajo Práctico Integrador

Food Store

Sistema de Gestión de Pedidos de Comida

Integrantes:

- Mateo Algañaraz
- Santiago Barretto
- Ignacio Rodríguez

Comisión: 3

Fecha de entrega: 22/06/2026

Índice

1. Marco teórico	3
2. Decisiones técnicas y arquitectura	4
2.1 Organización por paquetes	4
2.2 Separación por capas	4
2.3 Modelo de dominio y UML	5
2.4 Persistencia y baja lógica	6
2.5 Transacciones y rollback	6
2.6 Validaciones y reglas de negocio	6
3. Pruebas realizadas	7
4. Dificultades y soluciones	7
5. Conclusión	8
6. Bibliografía y herramientas	8
7. Enlaces de la entrega	8
8. Capturas	9

1. Marco Teórico

El presente trabajo se desarrolló utilizando Java 21 y el enfoque de Programación Orientada a Objetos (POO). A continuación se describen brevemente las tecnologías y conceptos clave aplicados.

1.1 Java y POO

Java es un lenguaje de programación orientado a objetos, multiplataforma y fuertemente tipado. Permite modelar el dominio del problema mediante clases, atributos, métodos y relaciones entre objetos (herencia, asociación, composición), tal como exige el modelo UML provisto en la consigna.

1.2 Colecciones (Collections Framework)

Las colecciones de Java, especialmente List y ArrayList, se utilizaron para representar relaciones entre objetos. Por ejemplo, una categoría puede contener productos, un usuario puede tener pedidos y un pedido contiene una lista de detalles. La información persistente del sistema se almacena en MySQL y se consulta mediante las clases DAO.

1.3 Interfaces

Una interfaz en Java define un contrato de comportamiento que las clases pueden implementar. En este proyecto se utiliza la interfaz Calculable, que obliga a la clase Pedido a definir el método calcularTotal(), estandarizando así la forma de obtener el total de un pedido a partir de sus detalles.

1.4 Manejo de Excepciones

El manejo de excepciones permite controlar errores en tiempo de ejecución sin que el programa se cierre de forma abrupta. En este trabajo se crearon excepciones propias (EntidadNoEncontradaException, StockInvalidoException, etc) que se lanzan y capturan en los puntos donde puede fallar una validación de negocio.

1.5 JDBC y bases de datos relacionales

JDBC (Java Database Connectivity) es la API estándar de Java que permite establecer conexiones y ejecutar operaciones sobre bases de datos relacionales. En este proyecto se utilizó MySQL como motor de base de datos y MySQL Connector/J como controlador JDBC.

Las operaciones de persistencia se encuentran centralizadas en las clases del paquete dao. Estas clases utilizan PreparedStatement para ejecutar instrucciones SQL parametrizadas, ResultSet para transformar los registros obtenidos en objetos Java y estructuras try-with-resources para cerrar automáticamente conexiones, sentencias y resultados.

La clase ConexionDB centraliza la apertura de conexiones y obtiene la URL, el usuario y la contraseña desde el archivo local db.properties. De esta manera, las credenciales no se repiten dentro de las clases DAO ni quedan escritas directamente en el código fuente.

La base de datos constituye la fuente persistente de información. Las colecciones presentes en algunas entidades se utilizan para representar relaciones del modelo, como los productos de una categoría, los pedidos de un usuario y los detalles de un pedido.

1.6 Maven

Maven se utilizó para administrar la estructura, compilación y dependencias del proyecto. El archivo pom.xml configura Java 21 e incorpora la dependencia mysql-connector-j, necesaria para conectar la aplicación con MySQL.

El uso de Maven permite descargar automáticamente las dependencias y ejecutar el proyecto de manera reproducible mediante los comandos:

```
mvn clean compile
```

```
mvn exec:java
```

2. Decisiones Técnicas y Arquitectura

2.1 Organización por paquetes

El proyecto fue desarrollado con Maven y organizado en paquetes según las responsabilidades de cada componente:

- config: contiene ConexionDB, responsable de cargar la configuración y entregar conexiones JDBC.
- entities: contiene las clases del dominio: Base, Categoria, Producto, Direccion, Usuario, Pedido, DetallePedido y la interfaz Calculable.
- enums: contiene los enumerados Rol, Estado y FormaPago.
- dao: contiene la interfaz genérica IBaseDAO y las clases que ejecutan sentencias SQL mediante JDBC.
- service: contiene validaciones, reglas de negocio y coordinación de operaciones.
- exception: contiene excepciones propias para representar errores del dominio y de persistencia.
- ui: contiene los menús de consola y la validación de entradas mediante Scanner.
- Main: crea los servicios y menús e inicia la aplicación.

2.2 Separación por capas

La aplicación sigue una arquitectura por capas. Main solamente configura e inicia el sistema. Las clases del paquete ui interactúan con el usuario y delegan las operaciones a los servicios.

Los servicios aplican las validaciones y reglas de negocio. Los DAO son los únicos componentes que contienen instrucciones SQL y trabajan directamente con JDBC. Las entidades representan el modelo de dominio y sus relaciones.

Esta separación evita mezclar la interacción por consola con la persistencia y facilita el mantenimiento del código.

2.3 Modelo de dominio y relaciones

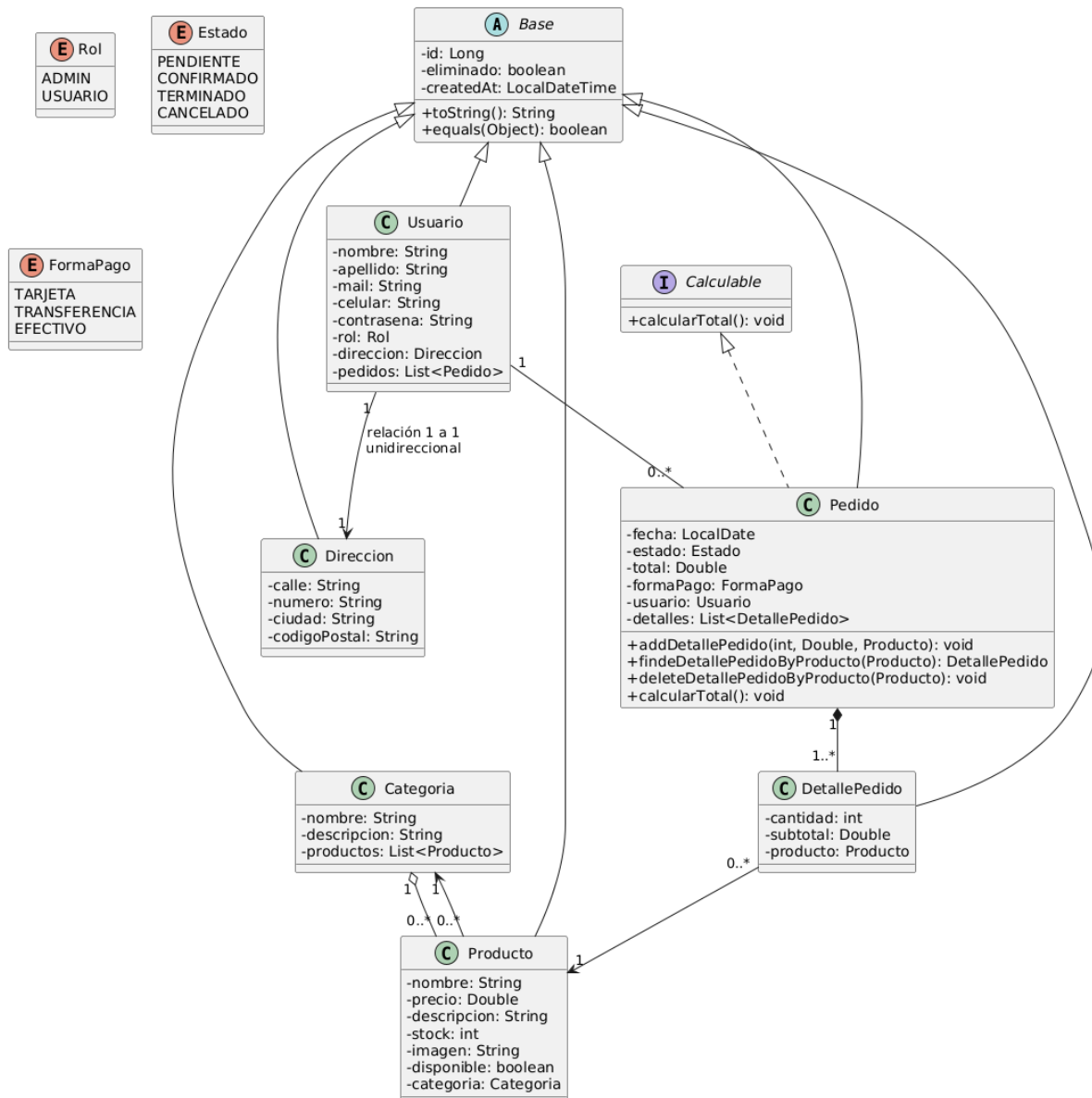
Todas las entidades heredan de la clase abstracta Base, que contiene los atributos comunes id, eliminado y createdAt.

Las principales relaciones son:

- Una categoría puede contener varios productos.
- Cada producto pertenece a una categoría.
- Un usuario puede tener varios pedidos.
- Un pedido contiene uno o más detalles.
- Cada detalle se relaciona con un producto.
- Cada usuario se relaciona de manera unidireccional con una única dirección.

La relación 1 a 1 entre Usuario y Direccion también se asegura en la base de datos mediante la restricción UNIQUE sobre usuario.direccion_id. De esta manera, una misma dirección no puede ser asignada a más de un usuario.

(Figura 1. Diagrama UML del sistema Food Store.)



La clase Pedido implementa la interfaz Calculable. Su método calcularTotal() suma los subtotales de los detalles activos.

2.4 Persistencia y baja lógica

El archivo database/schema.sql permite recrear completamente la base foodstore_db, sus tablas, claves foráneas, restricciones y datos de prueba.

Las eliminaciones se implementan mediante baja lógica. En lugar de ejecutar un DELETE, los DAO actualizan el campo eliminado a true. Los métodos de listado consultan solamente registros con eliminado = false.

Esta decisión conserva el historial y evita romper relaciones con pedidos y detalles existentes.

2.5 Transacciones y rollback

La creación de un pedido se ejecuta dentro de una única transacción JDBC. PedidoService obtiene una conexión y desactiva el modo automático mediante setAutoCommit(false).

La operación realiza los siguientes pasos:

1. Inserta la cabecera del pedido.
2. Obtiene y bloquea cada producto para su actualización.
3. Verifica que el producto exista, esté disponible y tenga stock suficiente.
4. Inserta cada detalle.
5. Descuenta el stock.
6. Calcula y actualiza el total.
7. Confirma la operación mediante commit().

Todos los DAO involucrados reciben la misma conexión. Si cualquier operación produce una excepción, se ejecuta rollback(), evitando que quede guardado un pedido incompleto o que se descuente stock incorrectamente.

2.6 Validaciones y reglas de negocio

El sistema valida, entre otros casos:

- Nombres y campos obligatorios no vacíos.
- Precio y stock mayores o iguales a cero.
- Cantidades de detalle mayores que cero.
- Existencia y estado activo de categorías, productos y usuarios.
- Mail de usuario único.
- Nombre de categoría único.
- Dirección no asignada previamente a otro usuario.
- Stock suficiente para confirmar pedidos.
- IDs existentes.
- Opciones numéricas válidas en los menús.

3. Pruebas realizadas

Prueba	Resultado obtenido
Crear categoría válida	Se guarda y muestra el ID
Crear categoría repetida	Se informa nombre duplicado
Crear producto con precio negativo	La operación es rechazada
Crear usuario con mail repetido	La operación es rechazada
Asignar una dirección ocupada	La relación 1:1 impide continuar
Crear pedido correcto	Se guardan cabecera y detalles, se descuenta stock
Crear pedido sin stock	Se ejecuta rollback
Eliminar una entidad	Se marca eliminado = true
Listar entidades	No se muestran registros eliminados

4. Dificultades y Soluciones

Una de las principales dificultades fue mantener separadas las responsabilidades de la interfaz, los servicios y el acceso a datos. Para solucionarlo, se estableció que las clases de menú solamente gestionaran entradas y mensajes, que los servicios concentraran las validaciones y que todo el código SQL permaneciera dentro de los DAO.

Otra dificultad fue implementar la creación de pedidos como una operación atómica. Un pedido requiere insertar la cabecera, guardar varios detalles y descontar el stock de diferentes productos. Esto se solucionó utilizando una única conexión JDBC con `setAutoCommit(false)`, confirmando los cambios mediante `commit()` y revirtiéndolos mediante `rollback()` ante cualquier error.

También fue necesario asegurar la relación uno a uno entre Usuario y Dirección. La solución se implementó tanto en Java, validando que una dirección no estuviera asignada, como en MySQL mediante una restricción `UNIQUE` sobre la clave foránea `direccion_id`.

La baja lógica presentó el desafío de conservar los registros históricos sin mostrarlos en las operaciones normales. Se resolvió incorporando el atributo `eliminado` en la clase `Base` y filtrando los listados con `eliminado = false`.

Finalmente, se centralizó la configuración de conexión en `ConexionDB` y en un archivo local `db.properties`, evitando repetir credenciales en las clases DAO y facilitando la configuración del proyecto en diferentes equipos.

5. Conclusión

El desarrollo de Food Store permitió aplicar Programación Orientada a Objetos, interfaces, herencia, colecciones, excepciones, arquitectura por capas, patrón DAO y persistencia mediante JDBC.

El sistema permite gestionar categorías, productos, direcciones, usuarios y pedidos desde una interfaz de consola. Además, incorpora validaciones, baja lógica, relaciones entre entidades y transacciones para garantizar la consistencia entre pedidos, detalles y stock.

La utilización de Maven y un script SQL único facilita la instalación y ejecución del proyecto desde cero.

6. Bibliografía / Webgrafía

- Oracle. Java SE 21 Documentation: <https://docs.oracle.com/en/java/javase/21/>
- Oracle. JDBC Basics: <https://docs.oracle.com/javase/tutorial/jdbc/>
- Apache Maven Documentation: <https://maven.apache.org/guides/>
- MySQL Reference Manual: <https://dev.mysql.com/doc/>
- MySQL Connector/J: <https://dev.mysql.com/doc/connector-j/en/>
- PlantUML: <https://plantuml.com/>
- ChatGPT: utilizado como herramienta de apoyo para revisión, explicación, organización del trabajo y para los ejemplos de la db. El código y el informe final fueron revisados y validados por los integrantes.

7. Enlaces de la Entrega

Recurso	Enlace
Repositorio del proyecto	https://github.com/Santiago-Btto/TPI-FINAL-PRO2
Documento PDF	Incluido dentro del ZIP en docs/Informe_TPI_FoodStore.pdf

8. Capturas

Menu principal:

```
=====
      SISTEMA DE PEDIDOS - FOOD STORE
=====
1. Categorías
2. Productos
3. Direcciones
4. Usuarios
5. Pedidos
0. Salir
Seleccione: |
```

Listado de productos:

```
=== PRODUCTOS ===
1. Listar
2. Crear
3. Editar
4. Eliminar
5. Listar por categoría
6. Buscar por nombre
0. Volver
Seleccione: 1
[ID: 1] Gaseosa           | $1800,00 | Stock: 20 | Disponible: Sí | Categoría: Bebidas
[ID: 2] Agua mineral     | $1200,00 | Stock: 30 | Disponible: Sí | Categoría: Bebidas
[ID: 3] Hamburguesa      | $6500,00 | Stock: 10 | Disponible: Sí | Categoría: Comidas
[ID: 4] Pizza            | $9000,00 | Stock: 8  | Disponible: Sí | Categoría: Comidas
[ID: 5] Flan             | $2500,00 | Stock: 12 | Disponible: Sí | Categoría: Postres
```

Usuario y su relación con dirección:

```
=====
      SISTEMA DE PEDIDOS - FOOD STORE
=====
1. Categorías
2. Productos
3. Direcciones
4. Usuarios
5. Pedidos
0. Salir
Seleccione: 3

=== DIRECCIONES (1-1 con usuario) ===
1. Listar
2. Crear
3. Editar
4. Eliminar
0. Volver
Seleccione: 2
Calle: Muslera
Número: 111
Ciudad: Godoy Cruz
Código postal: 5501
Dirección creada con ID 4
```

```
=====
      SISTEMA DE PEDIDOS - FOOD STORE
=====
1. Categorías
2. Productos
3. Direcciones
4. Usuarios
5. Pedidos
0. Salir
Seleccione: 4

=== USUARIOS ===
1. Listar
2. Crear
3. Editar
4. Eliminar
5. Buscar por mail
0. Volver
Seleccione: 2
[ID: 1] San Martín 1200, Mendoza (5500)
[ID: 2] Las Heras 350, Godoy Cruz (5501)
[ID: 3] Colón 89, Luján de Cuyo (5507)
[ID: 4] Muslera 111, Godoy Cruz (5501)
ID de dirección: 4
Nombre: Pepe
Apellido: Argento
Mail: pepe@gmail.com
Celular: 22132131
Contraseña: hola
Rol 1 = ADMIN, 2 = USUARIO: 2
Usuario creado con ID 3
```

Pedido creado:

```
=== PEDIDOS ===
1. Listar
2. Crear con detalles
3. Ver detalle
4. Actualizar estado y forma de pago
5. Eliminar
6. Listar por usuario
7. Demostrar rollback por stock insuficiente
0. Volver
Seleccione: 2
[ID: 1] Admin Sistema | Mail: admin@foodstore.com | Rol: ADMIN | Dirección: San Martín 1200
[ID: 2] Juan Pérez | Mail: juan@mail.com | Rol: USUARIO | Dirección: Las Heras 350
ID de usuario: 1
1 = TARJETA 2 = TRANSFERENCIA 3 = EFECTIVO
Forma de pago: 2
[ID: 1] Gaseosa | $1800,00 | Stock: 20 | Disponible: Sí | Categoría: Bebidas
[ID: 2] Agua mineral | $1200,00 | Stock: 30 | Disponible: Sí | Categoría: Bebidas
[ID: 3] Hamburguesa | $6500,00 | Stock: 10 | Disponible: Sí | Categoría: Comidas
[ID: 4] Pizza | $9000,00 | Stock: 8 | Disponible: Sí | Categoría: Comidas
[ID: 5] Flan | $2500,00 | Stock: 12 | Disponible: Sí | Categoría: Postres
ID de producto: 3
Cantidad: 1
Detalle agregado. Total provisorio: $6500,00
¿Agregar otro producto? (S/N): n
Pedido creado con ID 1 y total $6500.0
```

Error esperado ejemplo (stock):

```
=== PEDIDOS ===
1. Listar
2. Crear con detalles
3. Ver detalle
4. Actualizar estado y forma de pago
5. Eliminar
6. Listar por usuario
7. Demostrar rollback por stock insuficiente
0. Volver
Seleccione: 2
[ID: 1] Admin Sistema | Mail: admin@foodstore.com | Rol: ADMIN | Dirección: San Martín 1200
[ID: 2] Juan Pérez | Mail: juan@mail.com | Rol: USUARIO | Dirección: Las Heras 350
ID de usuario: 1
1 = TARJETA 2 = TRANSFERENCIA 3 = EFECTIVO
Forma de pago: 3
[ID: 1] Gaseosa | $1800,00 | Stock: 20 | Disponible: Sí | Categoría: Bebidas
[ID: 2] Agua mineral | $1200,00 | Stock: 30 | Disponible: Sí | Categoría: Bebidas
[ID: 3] Hamburguesa | $6500,00 | Stock: 9 | Disponible: Sí | Categoría: Comidas
[ID: 4] Pizza | $9000,00 | Stock: 8 | Disponible: Sí | Categoría: Comidas
[ID: 5] Flan | $2500,00 | Stock: 12 | Disponible: Sí | Categoría: Postres
ID de producto: 4
Cantidad: 40
Detalle agregado. Total provisorio: $360000,00
¿Agregar otro producto? (S/N): n
Error: Stock insuficiente para Pizza. Disponible: 8
```